Step 2 — A Real SQLite-Backed Store, and the Search Tool — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Two hints — work through them in order.

- Hint 1 — The pieces, and where each one lives
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The pieces, and where each one lives

Two real changes: the data becomes real (SQLite), and bad input gets rejected.

- `src/db.py` — all the SQL, behind small functions: `init_db(path)` (create the table if missing), `insert_note(path, text)` (returns the new id from `cursor.lastrowid`), `find_notes(path, query)` (a **case-insensitive** substring search — `LOWER(text) LIKE '%...%'`, and `query.lower()`).
- `src/exceptions.py` — add `EmptyNoteError(Exception)` next to Step 1's `MissingConfigError`.
- `server.py` — `add_note` now writes to the database **and validates**: strip the text, and if it's empty, reject it. `search_notes(query: str) -> list` is the new second tool.

The one real gotcha: **how the rejection reaches the client.** When a tool *raises*, the current `mcp` SDK hands the client only a generic *"Error executing tool add_note"* — no reason. So raise `EmptyNoteError` **and catch it**, returning a clear `f"Error: {e}"` string. That's Doc06's "get a tool's failure back to the model, correctly" — a clean result the client can read, not a crash and not a vague message.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

`add_note` is this shape — fill in the parts marked:

```
# server.py (add_note, now db-backed; search_notes is new)
from src.db import find_notes, init_db, insert_note
from src.exceptions import EmptyNoteError

DB_PATH = config.notes_db_path
init_db(DB_PATH)

@mcp.tool()
def add_note(text: str) -> str:
    """..."""
    try:
        cleaned = text.strip()
        if not cleaned:
            raise EmptyNoteError("note text cannot be empty or whitespace.")
        new_id = insert_note(DB_PATH, cleaned)
        logger.info(f"Note {new_id} added")
        return f"Note added with id {new_id}."
    except EmptyNoteError as e:
        logger.warning("Rejected an empty note")
        return f"Error: {e}"

@mcp.tool()
def search_notes(query: str) -> list:
    """..."""
    return find_notes(DB_PATH, query)
```

Write `find_notes` with `LOWER(text) LIKE ?` and `(f"%{query.lower()}%",)` so `"DENTIST"` matches `"dentist"`. Test it two ways: `tests/unit/test_db.py` (insert → find, case-insensitive, no-match) with no server, and extend `tests/integration/test_server.py` (search case, empty-rejection case). **Prove persistence by hand:** add notes, fully restart the server, and confirm `search_notes` still finds them. Compare against the Solution.